

Unity Thesis Document

K. Deepak Reddy | CSGD | Roll No. 13

Procedural Landmass Generation

Procedural Landmass Generation involves the use of algorithms to create natural-looking land formations such as continents, islands, and mountain ranges. By using noise functions, this method can generate large, varied, and realistic terrains without manual design. It is commonly used in games and simulations to create immersive and dynamically generated worlds.



BACKSTAGE PASS
INSTITUTE OF GAMING AND TECHNOLOGY

Submitted to

Backstage Pass Institute

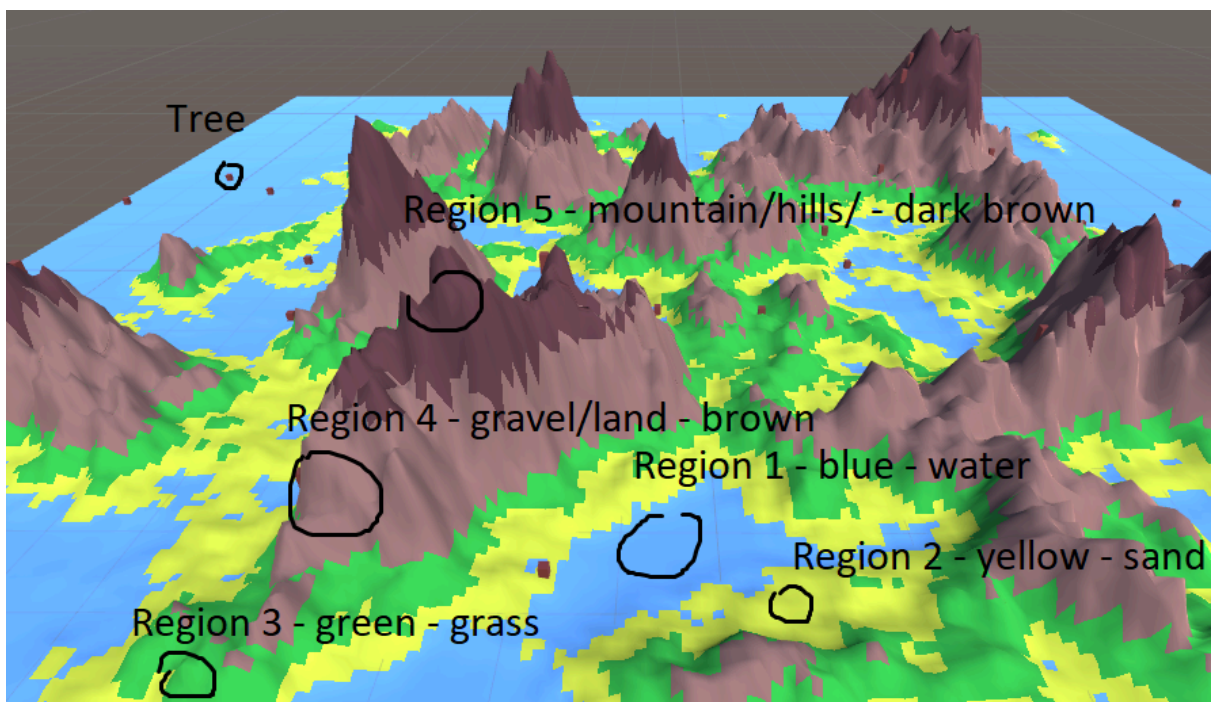
Lecturer in charge: **Mr. Sandeep Salimeda**

Abstract

This thesis presents the design and implementation of a **procedural landmass generation** system in Unity. Using **noise** functions, **color mapping**, and **mesh generation**, the system produces **natural-looking** terrains that can scale to infinite environments.

Introduction

The goal of this project is to design and implement a **procedural landmass generation system** in Unity that produces visually appealing and scalable terrains. The approach combines **noise-based heightmaps**, **biome color mapping**, and **mesh generation** to produce infinite landscapes.



Background

Procedural generation has long been used in games like Minecraft and **No Man's Sky**. Central to this process are noise functions such as **Perlin noise**, which simulate natural randomness. Unity provides a flexible environment for integrating these algorithms into interactive systems.



Design & Conceptual Model

The system architecture follows a modular pipeline: **Noise.cs** generates noise maps, **MapGenerator.cs** applies biome-based coloring, **TextureGenerator.cs** converts data into textures, **MeshGenerator.cs** builds terrain meshes, **MapDisplay.cs** renders outputs, and **EndlessTerrain.cs** manages chunk-based infinite terrain.

Noise Script

```
using UnityEngine;
```

```
public static class Noise
```

```
{  
    public static float[,] GenerateNoiseMap(int _width, int _height, int  
_seed, float _scale, int _octaves, float _persistence, float _lacunarity,  
Vector2 _offset)
```

```
{  
    float[,] _noiseMap = new float[_width, _height];
```

```
    System.Random _prng = new System.Random(_seed);  
    Vector2[] _octaveOffsets = new Vector2[_octaves];
```

```
    for(int i = 0; i < _octaves; i++)  
    {  
        float _offsetX = _prng.Next(-100000, 100000) + _offset.x;  
        float _offsetY = _prng.Next(-100000, 100000) + _offset.y;  
        _octaveOffsets[i] = new Vector2(_offsetX, _offsetY);  
    }
```

```
    float _maxNoiseHeight = float.MinValue;  
    float _minNoiseHeight = float.MaxValue;
```

```
    float _halfWidth = _width / 2f;
```

```

float _halfHeight = _height / 2f;

for (int y = 0; y < _height; y++)
{
    for(int x = 0; x < _width; x++)
    {
        float _amplitude = 1;
        float _frequency = 1;
        float _noiseHeight = 0;

        for (int i = 0; i < _octaves; i++)
        {
            float _sampleX = ((x - _halfWidth) / _scale -
            _octaveOffsets[i].x) * _frequency;
            float _sampleY = ((y - _halfHeight) / _scale -
            _octaveOffsets[i].y) * _frequency;

            float _noiseValue = Mathf.PerlinNoise(_sampleX, _sampleY)
            * 2 - 1;

            _noiseHeight += _noiseValue * _amplitude;

            _amplitude *= _persistance;
            _frequency *= _lacunarity;
        }

        if(_noiseHeight > _maxNoiseHeight)
            _maxNoiseHeight = _noiseHeight;
        else if(_noiseHeight < _minNoiseHeight)
            _minNoiseHeight = _noiseHeight;

        _noiseMap[x, y] = Mathf.InverseLerp(_minNoiseHeight,
        _maxNoiseHeight, _noiseHeight);
    }
}

```

```
        return _noiseMap;
    }
}
```

MapGenerator Script

```
using System;
using System.Collections.Generic;
using System.Threading;
using UnityEngine;
using static MapGenerator;

public enum DrawMode
{
    NoiseMap,
    ColorMap,
    Mesh
}

public class MapGenerator : MonoBehaviour
{
    [SerializeField] private DrawMode _drawMode;

    public const int _mapChunkSize = 241;
    [Range(0, 6)] [SerializeField] private int _levelOfDetail;
    [SerializeField] private float _noiseScale;

    [SerializeField] private int _octaves;
    [Range(0, 1)] [SerializeField] private float _persistance;
    [SerializeField] private float _lacunarity;

    [SerializeField] private int _seed;
    [SerializeField] private Vector2 _offset;
```

```

[SerializeField] private float _heightMultiplier;
[SerializeField] private AnimationCurve _meshHeightCurve;

public bool _autoUpdate;

private Queue<MapThreadInfo<MapData>>
_mapDataThreadInfoQueue = new Queue<MapThreadInfo<MapData>>();
private Queue<MapThreadInfo<MeshData>>
_meshDataThreadInfoQueue = new
Queue<MapThreadInfo<MeshData>>();

[SerializeField] private TerrainType[] _regions;

public void DrawMapInEditor()
{
    MapData _mapData = GenerateMapData();

    MapDisplay _mapDisplay =
FindFirstObjectByType<MapDisplay>();
    if (_drawMode == DrawMode.NoiseMap)

_mapDisplay.DrawTexture(TextureGenerator.TextureFromNoiseMap(_ma
pData._heightMap));
    else if (_drawMode == DrawMode.ColorMap)

_mapDisplay.DrawTexture(TextureGenerator.TextureFromColorMap(_ma
pData._colorMap, _mapChunkSize, _mapChunkSize));
    else if (_drawMode == DrawMode.Mesh)

_mapDisplay.DrawMesh(MeshGenerator.GenerateTerrainMesh(_mapData
._heightMap, _heightMultiplier, _meshHeightCurve, _levelOfDetail),
TextureGenerator.TextureFromColorMap(_mapData._colorMap,
_mapChunkSize, _mapChunkSize));
}

```

```

public void RequestMapData(Action<MapData> _callback)
{
    ThreadStart _threadStart = delegate
    {
        MapDataThread(_callback);
    };

    new Thread(_threadStart).Start();
}

public void MapDataThread(Action<MapData> _callback)
{
    MapData _mapData = GenerateMapData();

    lock(_mapDataThreadInfoQueue)
        _mapDataThreadInfoQueue.Enqueue(new
MapThreadInfo<MapData>(_callback, _mapData));
}

public void RequestMeshData(MapData _mapData, Action<MeshData>
_callback)
{
    ThreadStart _threadStart = delegate
    {
        MeshDataThread(_mapData, _callback);
    };
    new Thread(_threadStart).Start();
}

public void MeshDataThread(MapData _mapData, Action<MeshData>
_callback)
{
    MeshData _meshData =
MeshGenerator.GenerateTerrainMesh(_mapData._heightMap,
_heightMultiplier, _meshHeightCurve, _levelOfDetail);
}

```



```

        lock(_meshDataThreadInfoQueue)
            _meshDataThreadInfoQueue.Enqueue(new
MapThreadInfo<MeshData>(_callback, _meshData));
    }

    private void Update()
    {
        if(_mapDataThreadInfoQueue.Count > 0)
        {
            for (int i = 0; i < _mapDataThreadInfoQueue.Count; i++)
            {
                MapThreadInfo<MapData> _threadInfo =
_mapDataThreadInfoQueue.Dequeue();
                _threadInfo._callback(_threadInfo._parameter);
            }
        }

        if (_meshDataThreadInfoQueue.Count > 0)
        {
            for (int i = 0; i < _meshDataThreadInfoQueue.Count; i++)
            {
                MapThreadInfo<MeshData> _threadInfo =
_meshDataThreadInfoQueue.Dequeue();
                _threadInfo._callback(_threadInfo._parameter);
            }
        }
    }

    private MapData GenerateMapData()
    {
        float[, ] _noiseMap = Noise.GenerateNoiseMap(_mapChunkSize,
_mapChunkSize, _seed, _noiseScale, _octaves, _persistance, _lacunarity,
_offset);
    }

```

```

Color[] _colorMap = new Color[_mapChunkSize * _mapChunkSize];
for(int y = 0; y < _mapChunkSize; y++)
{
    for(int x = 0; x < _mapChunkSize; x++)
    {
        float _currentHeight = _noiseMap[x, y];

        for(int i = 0; i < _regions.Length; i++)
        {
            if(_currentHeight <= _regions[i]._height)
            {
                _colorMap[y * _mapChunkSize + x] = _regions[i]._color;
                break;
            }
        }
    }
}

return new MapData(_noiseMap, _colorMap);
}

private void OnValidate()
{
    if (_noiseScale <= 0)
        _noiseScale = 0.01f;

    if(_lacunarity < 1)
        _lacunarity = 1;

    if(_octaves < 0)
        _octaves = 0;
}

public struct MapThreadInfo<T>
{

```

```

    public readonly Action<T> _callback;
    public readonly T _parameter;

    public MapThreadInfo(Action<T> _callback, T _parameter)
    {
        this._callback = _callback;
        this._parameter = _parameter;
    }
}

```

```

[System.Serializable]
public struct TerrainType
{
    public string _name;
    public float _height;
    public Color _color;
}

```

```

public struct MapData
{
    public readonly float[,] _heightMap;
    public readonly Color[] _colorMap;

    public MapData(float[,] _heightMap, Color[] _colorMap)
    {
        this._heightMap = _heightMap;
        this._colorMap = _colorMap;
    }
}

```

TextureGenerator Script

```

using UnityEngine;

```

```

public static class TextureGenerator
{
    public static Texture2D TextureFromColorMap(Color[] _colorMap, int
_mapWidth, int _mapHeight)
    {
        Texture2D texture = new Texture2D(_mapWidth, _mapHeight);
        texture.filterMode = FilterMode.Point;
        texture.wrapMode = TextureWrapMode.Clamp;
        texture.SetPixels(_colorMap);
        texture.Apply();
        return texture;
    }

    public static Texture2D TextureFromNoiseMap(float[,] _noiseMap)
    {
        int _mapWidth = _noiseMap.GetLength(0);
        int _mapHeight = _noiseMap.GetLength(1);

        Color[] _colorMap = new Color[_mapWidth * _mapHeight];

        for (int y = 0; y < _mapHeight; y++)
        {
            for (int x = 0; x < _mapWidth; x++)
                _colorMap[y * _mapWidth + x] = Color.Lerp(Color.black,
Color.white, _noiseMap[x, y]);
        }

        return TextureFromColorMap(_colorMap, _mapWidth, _mapHeight);
    }
}

```

MeshGenerator Script

```

using System.Runtime.CompilerServices;
using UnityEngine;

```

```

public static class MeshGenerator
{
    public static MeshData GenerateTerrainMesh(float[, ] _heightMap, float
_heightMultiplier, AnimationCurve _meshHeightCurve, int
_levelOfDetail)
    {
        AnimationCurve _heightCurve = new
AnimationCurve(_meshHeightCurve.keys);
        int _width = _heightMap.GetLength(0);
        int _height = _heightMap.GetLength(1);
        float _topLeftX = (_width - 1) / -2f;
        float _topLeftZ = (_height - 1) / 2f;

        int _meshSimplificationIncrement = (_levelOfDetail == 0)? 1 :
_levelOfDetail * 2;
        int _verticesPerLine = (_width - 1)/_meshSimplificationIncrement +
1;

        MeshData _meshData = new MeshData(_width, _height);
        int _vertexIndex = 0;

        for (int y = 0; y < _height; y += _meshSimplificationIncrement)
        {
            for(int x = 0; x < _width; x += _meshSimplificationIncrement)
            {
                _meshData._vertices[_vertexIndex] = new Vector3(_topLeftX +
x, _heightCurve.Evaluate(_heightMap[x, y]) * _heightMultiplier,
_topLeftZ - y);
                _meshData._uvs[_vertexIndex] = new Vector2(x / (float)_width,
y / (float)_height);

                if (x < _width - 1 && y < _height - 1)
                {

```

```

        _meshData.AddTriangle(_vertexIndex, _vertexIndex +
        _verticesPerLine + 1, _vertexIndex + _verticesPerLine);
        _meshData.AddTriangle(_vertexIndex + _verticesPerLine + 1,
        _vertexIndex, _vertexIndex + 1);
    }

    _vertexIndex++;
}
}

return _meshData;
}
}

```

```

public class MeshData
{
    public Vector3[] _vertices;
    public int[] _triangles;
    public Vector2[] _uvs;

    private int _triangleIndex;

    public MeshData(int _meshWidth, int _meshHeight)
    {
        _vertices = new Vector3[_meshWidth * _meshHeight];
        _triangles = new int[( _meshWidth - 1) * ( _meshHeight - 1) * 6];
        _uvs = new Vector2[_meshWidth * _meshHeight];
    }

    public void AddTriangle(int _a, int _b, int _c)
    {
        _triangles[_triangleIndex] = _a;
        _triangles[_triangleIndex + 1] = _b;
        _triangles[_triangleIndex + 2] = _c;
    }
}

```

```

        _triangleIndex += 3;
    }

    public Mesh CreateMesh()
    {
        Mesh _mesh = new Mesh();
        _mesh.vertices = _vertices;
        _mesh.triangles = _triangles;
        _mesh.uv = _uvs;
        _mesh.RecalculateNormals();
        return _mesh;
    }
}

```

MapDisplay Script

```

using UnityEngine;

public class MapDisplay : MonoBehaviour
{
    [SerializeField] private Renderer _textureRenderer;
    [SerializeField] private MeshFilter _meshFilter;
    [SerializeField] private MeshRenderer _meshRenderer;

    public void DrawTexture(Texture2D _texture)
    {
        _textureRenderer.sharedMaterial.mainTexture = _texture;
        _textureRenderer.transform.localScale = new Vector3(_texture.width,
1, _texture.height);
    }

    public void DrawMesh(MeshData _meshData, Texture2D _texture)
    {
        _meshFilter.sharedMesh = _meshData.CreateMesh();
    }
}

```

```

        _meshRenderer.sharedMaterial.mainTexture = _texture;
    }
}

```

EndlessTerrain Script

```

using UnityEngine;
using System.Collections.Generic;

public class EndlessTerrain : MonoBehaviour
{
    [SerializeField] private Transform _viewer;
    [SerializeField] private Material _material;
    public const float _maxViewDistance = 450;
    public static Vector2 _viewerPosition;

    private static MapGenerator _mapGenerator;

    private int _chunkSize;
    private int _chunksVisibleInViewDistance;

    private Dictionary<Vector2, TerrainChunk> _terrainChunkDictionary =
new Dictionary<Vector2, TerrainChunk>();
    private List<TerrainChunk> _terrainChunksVisibleLastUpdate = new
List<TerrainChunk>();

    private void Start()
    {
        _mapGenerator = FindFirstObjectByType<MapGenerator>();
        _chunkSize = MapGenerator._mapChunkSize - 1;
        _chunksVisibleInViewDistance =
Mathf.RoundToInt(_maxViewDistance / _chunkSize);
    }
}

```



```

private void Update()
{
    _viewerPosition = new Vector2(_viewer.position.x,
_viewer.position.z);
    UpdateVisibleChunks();
}

private void UpdateVisibleChunks()
{
    for(int i = 0; i < _terrainChunksVisibleLastUpdate.Count; i++)
    {
        _terrainChunksVisibleLastUpdate[i].SetVisible(false);
    }
    _terrainChunksVisibleLastUpdate.Clear();

    int _currentChunkCoordX = Mathf.RoundToInt(_viewerPosition.x /
_chunkSize);
    int _currentChunkCoordY = Mathf.RoundToInt(_viewerPosition.y /
_chunkSize);

    for(int yOffset = -_chunksVisibleInViewDistance; yOffset <=
_chunksVisibleInViewDistance; yOffset++)
    {
        for(int xOffset = -_chunksVisibleInViewDistance; xOffset <=
_chunksVisibleInViewDistance; xOffset++)
        {
            Vector2 _viewedChunkCoord = new
Vector2(_currentChunkCoordX + xOffset, _currentChunkCoordY +
yOffset);

            if(_terrainChunkDictionary.ContainsKey(_viewedChunkCoord))
            {

                _terrainChunkDictionary[_viewedChunkCoord].UpdateTerrainChunk();

```

```

        if(_terrainChunkDictionary[_viewedChunkCoord].IsVisible())
        {

            _terrainChunksVisibleLastUpdate.Add(_terrainChunkDictionary[_viewed
            ChunkCoord]);
        }
        else
        {
            _terrainChunkDictionary.Add(_viewedChunkCoord, new
            TerrainChunk(_viewedChunkCoord, _chunkSize, transform, _material));
        }
    }
}
}

```

```

public class TerrainChunk
{
    GameObject _meshObject;
    Vector2 _position;
    Bounds _bounds;

    MeshRenderer _meshRenderer;
    MeshFilter _meshFilter;

    public TerrainChunk(Vector2 _coord, int _size, Transform _parent,
    Material _material)
    {
        _position = _coord * _size;
        _bounds = new Bounds(_position, Vector2.one * _size);
        Vector3 _positionV3 = new Vector3(_position.x, 0, _position.y);

        _meshObject = new GameObject("Terrain Chunk");
        _meshRenderer =
        _meshObject.AddComponent<MeshRenderer>();
    }
}

```

```

    _meshFilter = _meshObject.AddComponent<MeshFilter>();
    _meshRenderer.material = _material;

    _meshObject.transform.position = _positionV3;
    _meshObject.transform.parent = _parent;
    SetVisible(false);

    _mapGenerator.RequestMapData(OnMapDataReceived);
}

public void OnMapDataReceived(MapData _mapData)
{
    _mapGenerator.RequestMeshData(_mapData,
OnMeshDataReceived);
}

public void OnMeshDataReceived(MeshData _meshData)
{
    _meshFilter.mesh = _meshData.CreateMesh();
}

public void UpdateTerrainChunk()
{
    float _viewerDistanceFromNearestEdge =
Mathf.Sqrt(_bounds.SqrDistance(_viewerPosition));
    bool _isVisible = _viewerDistanceFromNearestEdge <=
_maxViewDistance;
    SetVisible(_isVisible);
}

public void SetVisible(bool _isVisible)
{
    _meshObject.SetActive(_isVisible);
}

```

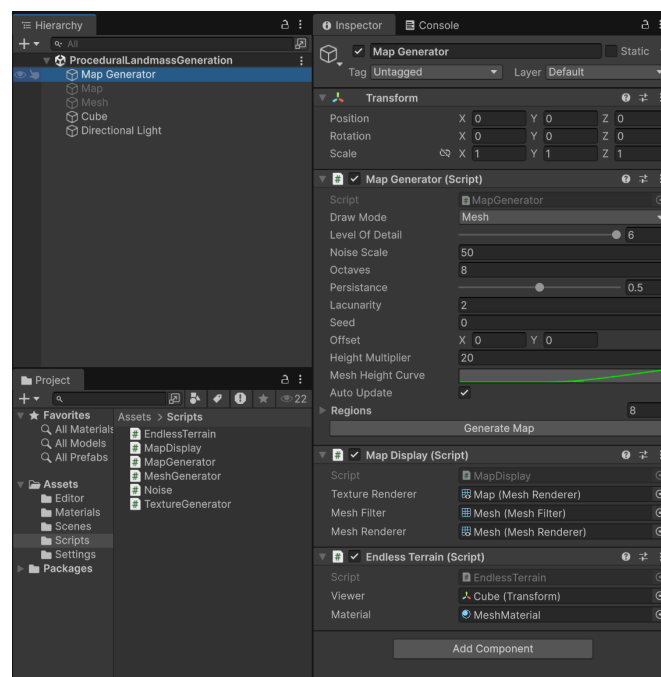
```

public bool IsVisible()
{
    return _meshObject.activeSelf;
}
}
}

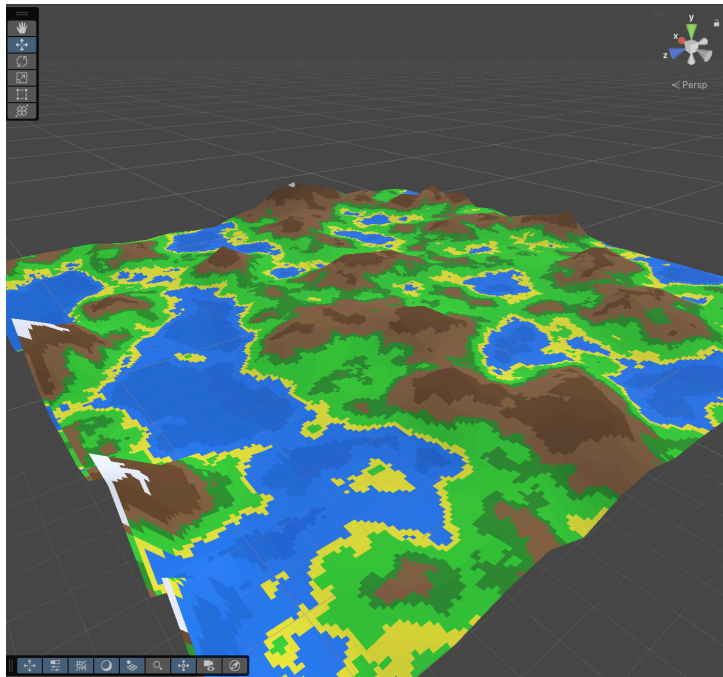
```

Implementation

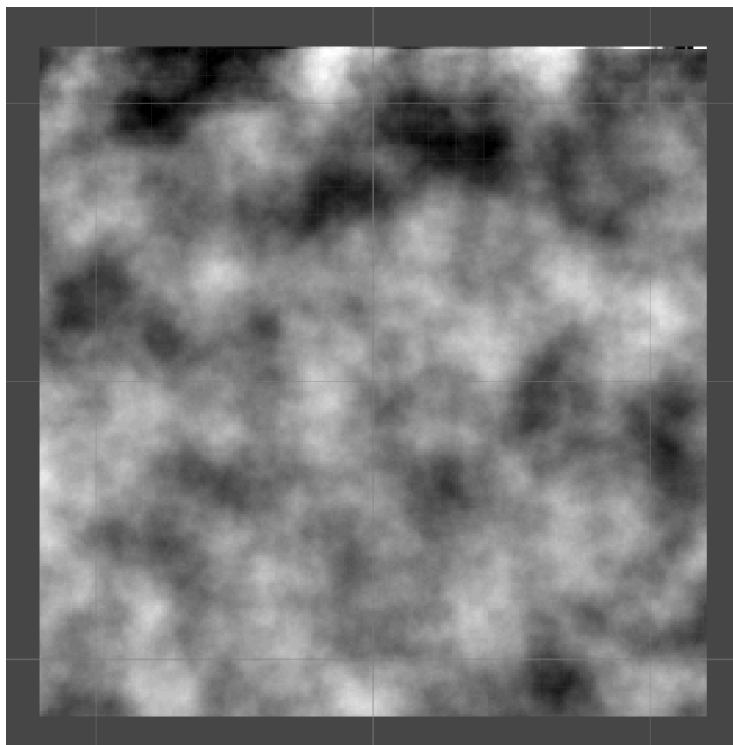
Unity Inspector Setup:



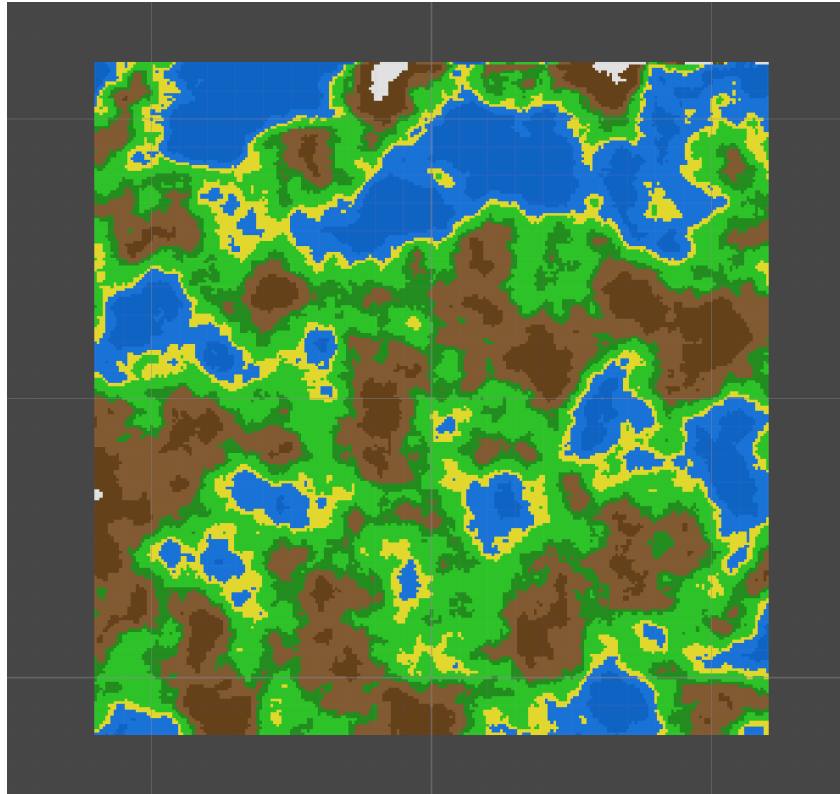
Generated Mesh Terrain:



Noise Map:



Color Map:



Evaluation

The system generated consistent terrains with diverse biomes. Infinite terrain loading worked smoothly within defined view distances. **Biome coloring** enhanced terrain **readability**. **Performance** was stable on **mid-level hardware**.

Lessons Learned

Key lessons include the importance of modular architecture, the effect of parameter tuning on realism, and the need for careful chunk management in infinite terrains.

Conclusion

The **procedural landmass generator** successfully produces infinite, noise-based terrains with **biome coloring** and **mesh rendering** in Unity. The system is **flexible** and **scalable**, making it useful for game developers and researchers alike.